

EE272 Homework 3

CNN Accelerator Part 2

Report

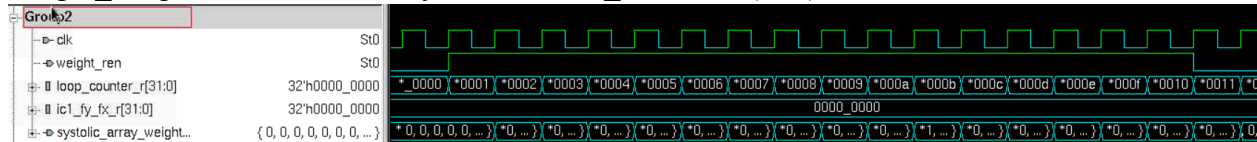
Student Name 1: Berk Gokmen
Student Name 2: Edison Altamirano

Please answer the questions below in the space provided and include the code/reports indicated in your submission.

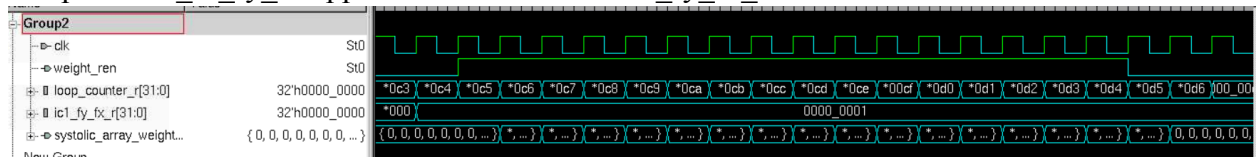
3.3 Convolution Controller

1. When is the weight double buffer read enabled (weight_ren)? Why is it enabled at these times? (5 points)

Weight_ren pulse last about 16 cycles ARRAY_HEIGHT (=16)



The pulse ic1_fy_fx_r happens at the start each ic1_fy_fx_block

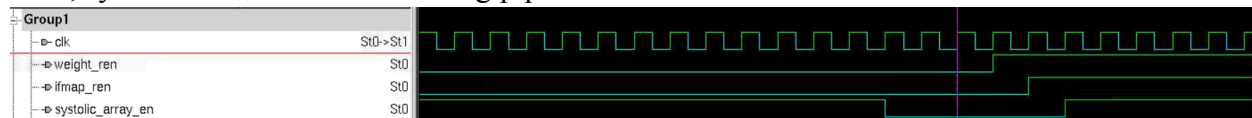


weight_ren is asserted during the compute phase (INNER_LOOP) in a short window at the beginning of each ic1_fy_fx iteration. It stays high for approximately ARRAY_HEIGHT (=16) cycles to read a full block of weights from the weight double buffer and load them into the systolic array rows. This limits weight reads to only the cycles needed for weight loading and keeps the array schedule aligned.

In conclusion, weight_ren turns on in short bursts at the start of each compute block (each ic1_fy_fx step). It stays on for about 16 cycles because the array has 16 rows, so it needs about 16 cycles to load one full set of weights into the systolic array. It's enabled only at those times to load the weights once, then it turns off to avoid reading extra weights while the array is just doing the math.

2. When is the input double buffer read enabled (ifmap_ren)? Why is it enabled at these times? (5 points)

As shown in the waveform, ifmap_ren is asserted shortly after weight_ren, following a one-cycle offset. This matches the controller behavior where the input (ifmap) double buffer is enabled one cycle after the weight double buffer, allowing time for weights to be read from the buffer and aligned through internal registers. By the time systolic_array_en is asserted, both weights and input activations are available at the MACs, ensuring the systolic array starts computing with valid, synchronized data and avoiding pipeline bubbles.



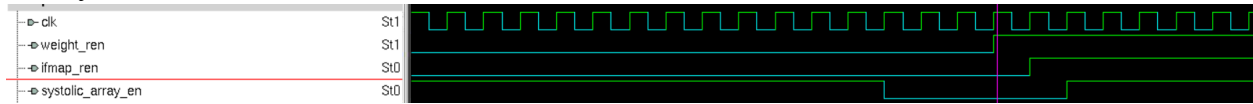
In the waveform, state_r is mostly 2 (INNER_LOOP), which is the compute phase where loop_counter_r advances. During this phase, ifmap_ren is asserted for loop_counter_r values from 1 up to IC1_FY_FX_OY0_OX0_c, matching the condition in the code. When state_r briefly switches to 3 (RESET_INNER_LOOP), the controller resets loop_counter_r back to 0, and ifmap_ren deasserts because the accelerator is transitioning between tiles rather than streaming inputs.



In conclusion, ifmap_ren is enabled during INNER_LOOP (2 state_r) when loop_counter_r is in the main compute range, the controller turns ifmap_ren on to read input data, and it keeps systolic_array_en on so the array can compute. When the controller leaves compute (state changes to reset/transition), the counter resets and those signals drop, meaning “we’re not computing right now, so stop reading inputs.”

3. When is the systolic array enabled (systolic_array_en)? Why do we wait to enable it? (5 points)

The signal `systolic_array_en` starts after `weight_ren/ifmap_ren` due to buffer + skew pipeline latency.



In the waveform, we can see that the `systolic_array_en` turns on at `loop_counter_r >= 2` because the controller waits a couple of cycles for the weights and inputs to reach the MACs. It stays on past the main loop (`IC1_FY_FX_OY0_OX0_c`) for extra `ARRAY_WIDTH + ARRAY_HEIGHT` cycles to let the systolic pipeline “flush” so the last results can finish coming out.

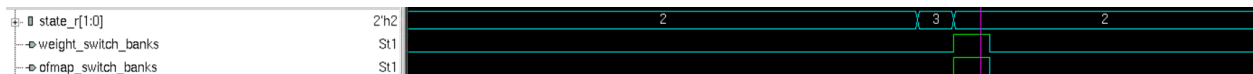


In conclusion, the array starts computing a couple cycles after we begin reading data, because the data needs time to arrive. It keeps running a little longer at the end so the last computations can finish and flow out of the pipeline.

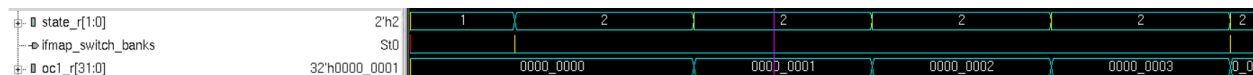
4. At what state of the finite state machine do we switch the input buffer banks? Why don't we switch it at the same time as the weight and output buffers? (5 points)

We switch the input buffer during the reset step, but only after we've finished all the output-channel blocks, because we reuse the same input data for several outputs. We switch weights and output every time, but we keep the input the same until we're done with that reuse.

When state_r briefly goes to 3 (RESET_INNER_LOOP), we see weight_switch_banks and ofmap_switch_banks pulse every time we hit that reset state (basically once per output tile). This shows weights and output buffers switch on each tile boundary.



In the waveform, bank switching happens during the reset phase of the FSM (state_r = 3, RESET_INNER_LOOP). We can see oc1_r counting up across multiple compute phases (state_r = 2), while ifmap_switch_banks stays low, and it only pulses when oc1_r finishes and wraps back to 0. This shows the input tile is reused across OC1 blocks, so the input bank is switched only at the OC1 boundary, not every reset.



5. Why is the product of IC1, FY, FX, OY0 and OX0 sent as a parameter into the convolution controller? How is it used in determining when systolic_array_en is high? (5 points)

The product $IC1 \cdot FY \cdot FX \cdot OY0 \cdot OX0$ is passed into the controller as `IC1_FY_FX_OY0_OX0_c` so the controller has a single “total work length” value for the inner loop without recomputing a large multiplication in hardware. In the waveform, `IC1_FY_FX_OY0_OX0_c` stays constant (0x1b90) while `loop_counter_r` counts up, and `systolic_array_en` remains high during this counting window; when the counter reaches the end of the work (and the controller transitions/reset occurs, causing `loop_counter_r` to wrap back to 0), `systolic_array_en` drops and then reasserts for the next iteration.

In the waveform, `IC1_FY_FX_OY0_OX0_c` is constant at 0x1b90, while `loop_counter_r` approaches the end of the window (0x1bb1) because the condition states that `loop_counter_r` should be less than `IC1_FY_FX_OY0_OX0_c + ARRAY_WIDTH(4) + ARRAY_HEIGHT(4)`, the `systolic_array_en` deasserts as the controller finishes the compute/flush period. After the reset, `loop_counter_r` wraps back to small values (0x0002), and `systolic_array_en` asserts again because the condition `loop_counter_r >= 2` is satisfied for the next iteration.



4 Integration Testing

1. Please paste a screenshot of the autograder passing all layers. We will also run this ourselves. (40 points)

```
[bgokmen@iron-02 dnn-accelerator-top-rtl]$ python3 autograder.py conv_tb -l all
Tests being run: ['test_conv_tb']
Running test_conv_tb with layer params: ./layers/resnet_conv1_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv2_x_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv3_1_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv3_x_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv4_1_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv4_x_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv5_1_params.json
Test passed!

Running test_conv_tb with layer params: ./layers/resnet_conv5_x_params.json
Test passed!

Tests passed: 8
Tests run: 8
```

5 Evaluating Accelerator Performance

1. The memory sizes you tested with (use a 16x16 systolic array). (5 points)

Buffer Type	Bank Depth	Word Width (Bits)	Total Size (Bytes)
Weight Buffer	8192	128 (16x8-bit)	256 KB
Ifmap Buffer	3600	128 (16x8-bit)	112.5 KB
Ofmap Buffer	256	512 (16x32-bit)	32 KB

Table 1. Table detailing the memory sizes used for testing the 16x16 systolic array.

Use a table to answer the questions below (based on layer)

2. The tiling you used for each ResNet-18 layer, meaning the values for IC0, OC0, OX0, OY0, FX, FY, IC1, OC1, OX1 and OY1.
3. The number of cycles your accelerator takes to run each ResNet-18 layer.
4. The ideal number of cycles your accelerator should take for each ResNet-18 layer.
5. The ratio of (3) and (4) above, meaning the utilization of your MAC units.
6. For every layer, write what determines its performance—is it compute bound, or input write bandwidth bound, or weight write bandwidth bound, or output write back bandwidth bound. You should determine this analytically, and then match it with what you observe in the waveforms.
7. Percentage utilized read and write bandwidth of each buffer, that is, if your total runtime for a layer is 10 cycles, and of those you read your input double buffer for 3 cycles, then your utilized read bandwidth for input double buffer is 3 / 10.

(20 points)

Layer	IC0, OC0, OX0, OY0, IC1, OC1, OX1, OY1, FX, FY	Actual Cycles	Ideal Cycles	MAC Util	Bottleneck	Ifmap Read BW	Ifmap Write BW	Weight Read BW	Weight Write BW	Ofmap Read BW	Ofmap Read WB BW	Ofmap Write BW
Conv1	16, 16, 14, 14, 1, 4, 8, 8, 7, 7	2,475,377	2,458,624	99.32%	Compute	99.45%	2.89%	8.12%	8.17%	97.42%	2.03%	99.45%
Conv2_x	16, 16, 14, 14, 4, 4, 4, 4, 3, 3	461,165	451,584	97.92%	Compute	98.60%	3.95%	8.05%	8.24%	95.84%	2.72%	98.60%
Conv3_1	16, 16, 7, 7, 4, 8, 4, 4, 3, 3	320,333	225,792	70.49%	Input Write	70.73%	4.84%	23.10%	23.26%	68.76%	1.96%	70.72%
Conv3_x	16, 16, 7, 7, 8, 8, 4, 4, 3, 3	595,517	451,584	75.83%	Weight Write	75.96%	1.88%	24.80%	24.99%	74.90%	1.05%	75.96%
Conv4_1	16, 16, 7, 7, 8, 16, 2, 2, 3, 3	313,437	225,792	72.04%	Input Write	72.29%	2.93%	23.61%	23.95%	71.27%	1.00%	72.28%
Conv4_x	16, 16, 7, 7, 16, 16, 2, 2, 3, 3	599,997	451,584	75.26%	Weight Write	75.40%	1.11%	24.62%	24.99%	74.86%	0.52%	75.39%
Conv5_1	16, 16, 7, 7, 16, 32, 1, 1, 3, 3	315,389	225,792	71.59%	Input Write	71.84%	2.35%	23.46%	24.17%	71.33%	0.50%	71.83%
Conv5_x	16, 16, 7, 7, 32, 32, 1, 1, 3, 3	609,149	451,584	74.13%	Weight Write	74.26%	0.88%	24.25%	25.00%	74.00%	0.26%	74.26%

Table 2. Table display loop tiling, cycles, utilization, and bottleneck values for each layer

The table below shows our analytical explanation for the bottlenecks for each layer. We calculated the 4 Tile sizes based on the loop bound parameters for each layer. The largest tile size determines the cycle count for that block, which corresponds to the bottleneck.

- **TILE_PE:** $OX0 \times OY0 \times IC1 \times FX \times FY$
- **TILE_IB:** $(IC0 \times IX0 \times IY0 \times IC1) / 4$
- **TILE_WB:** $(IC0 \times OC0 \times FX \times FY \times IC1) / 4$
- **TILE_OB:** $(OX0 \times OY0 \times OC0) / 1$

Layer	TILE_PE	TILE_IB	TILE_WB	TILE_OB	Bound
Conv1	9,604	4,356	3,136	3,136	Compute
Conv2_x	7,056	4,096	2,304	3,136	Compute
Conv3_1	1,764	3,600	2,304	784	Input
Conv3_x	3,528	2,592	4,608	784	Weight
Conv4_1	3,528	7,200	4,608	784	Input
Conv4_x	7,056	5,184	9,216	784	Weight
Conv5_1	7,056	14,400	9,216	784	Input
Conv5_x	14,112	10,368	18,432	784	Weight

Table 3. Table displaying analytical values for each layer

The table below shows signal counts from the waveforms, which also support the bounds identified previously:

- **Conv1 and Conv2_x layers:** These layers are compute-bound because the computation time (TILE_PE) significantly exceeds the time needed to fetch input (TILE_IB) or weight (TILE_WB) tiles. For Conv1, the TILE_PE is 9,604 cycles, which is more than double the 4,356 cycles needed for input loading. Waveform counts verify this efficiency, showing systolic_array_en is active for 2,469,698 out of 2,475,377 total cycles (99.32% utilization).
- **Conv3_x, Conv4_x, Conv5_x layers:** Increasing channel depth and decreasing spatial dimensions in deeper layers cause weight fetch time (TILE_WB) to surpass compute time (TILE_PE). For Conv5_x, the weight fetch time of 18,432 cycles exceeds the 14,112 cycles of computation. Waveforms confirm this stall, as systolic_array_en is active for only 453,362 cycles out of 609,149 (74% utilization). 152,261 weight_wen pulses are required to fill the buffers.
- **Conv3_1, Conv4_1, Conv5_1 layers:** For example, for Conv5_1, the analytical TILE_IB is 14,400 cycles compared to 7,056 cycles for computation. Waveform counts highlight this bottleneck: systolic_array_en is active for only 227,570 cycles within a 315,389 cycle total (71.59% utilization), showing the array is idle for nearly 30% of the runtime.

Layer	Cycles	systolic_array_en	ifmap_wen	ifmap_ren	weight_wen	weight_ren	ofmap_wen	ofmap_ren	ofmap_wb_ren
Conv1	2,475,377	2,469,698	71,570	2,461,763	202,272	200,964	2,461,731	2,411,391	50,176
Conv2_x	461,165	456,706	18,193	454,723	38,016	37,124	454,691	441,983	12,544
Conv3_1	320,333	230,546	15,497	226,579	74,501	73,988	226,547	220,258	6,272
Conv3_x	595,517	456,338	11,213	452,371	148,805	147,716	452,339	446,050	6,272
Conv4_1	313,437	228,562	9,197	226,579	75,077	73,988	226,547	223,394	3,136
Conv4_x	599,997	454,354	6,677	452,371	149,957	147,716	452,339	449,186	3,136
Conv5_1	315,389	227,570	7,397	226,579	76,229	73,988	226,547	224,962	1,568
Conv5_x	609,149	453,362	5,381	452,371	152,261	147,716	452,339	450,754	1,568

Table 4. Table displaying signal counts from waveforms for each layer

8. Suggest concrete ways for improving the performance of the accelerator while keeping the number of MAC units (16×16) and total memory size the same, and without drastically increasing the I/O bandwidth. (For example, you may decide to keep the total number of I/Os the same but redistribute their allocation between inputs, weights, outputs, and configuration.) Suggest concrete ways for reducing the amount of SRAM used without sacrificing performance. If you implement these improvements, you will receive extra credit. In that case turn in both your baseline accelerator and the improved version, and clearly document the difference between the two and quantify the improvement in the report. (10 points)

Compute-bound

Here the goal is to reduce fixed overhead and internal bubbles (not raw I/O bandwidth). A concrete improvement is to use larger tiles (increase $OX0 \times OY0$) while keeping the same total SRAM, because larger tiles amortize the pipeline fill/flush cost over more useful MAC work. Another compatible idea is quantization so more effective data fits per tile (same SRAM bytes hold more values), which can allow larger $OX0 \times OY0$ or fewer reloads without changing the physical memory size.

Input-bound

The goal is to fill the input double buffer faster or reload it less often without increasing total pins. A concrete fix is to redistribute the fixed I/O budget toward inputs: increase *IFMAP_FIFO_WORDS* while decreasing *WEIGHT_FIFO_WORDS* so the total I/O width stays roughly constant. In addition, use more aggressive prefetching: while the accelerator computes from bank A, start writing the next ifmap tile into bank B as early as possible so *ifmap_switch_banks* does not create a stall when the current tile finishes.

Weight-bound

The goal is to load weights faster or reuse them longer. Similarly, we can redistribute the same total I/O bandwidth toward weights: increase *WEIGHT_FIFO_WORDS* and reduce *IFMAP_FIFO_WORDS* if inputs are not the limiter, keeping the overall pin count/width essentially unchanged. Also apply weight prefetching into the free bank during compute so when *weight_switch_banks* occurs, the next weight tile is already resident and the systolic array does not drop *systolic_array_en* due to weight-loading bubbles.

Quantization (Improving all memory bounds):

One way to improve all memory bounds is through quantization. Reducing precision from 8-bit to 4-bit doubles I/O bandwidth and buffer capacity within the same physical constraints. This directly addresses the bandwidth bottlenecks in layers *Conv5_x*, which was weight bound at 74.13% utilization, and *Conv3_1*, which was input bound at 70.49% utilization. By halving the bit-width, the same 256 KB weight buffer and 112.5 KB input buffer can hold twice as many elements, allowing for larger tiling parameters that reduce frequency of reloads.

9. If you were successful in doing some extra credit work, please describe that here. (Points only if all new and existing tests pass, and new tests from you sufficiently test the new functionality.)